

Lab 1: Fundamental R Computing, Data Visualization & Optimization

Overview & Learning Objectives:

This laboratory module covers core programming and mathematical operations in R. Topics include vector/matrix manipulation, custom functions, image processing using grayscale intensity maps, hand-written digit recognition analysis, and numerical optimization via Newton-Raphson, Steepest Ascent, and Gradient Ascent algorithms.

1. Vector Operations & Matrix Manipulations

Q1.a Normal Distribution Sampling & Empirical Evaluation

Generate a random vector $\mathbf{v}$ of length $N = 100$ drawn from a normal distribution $\mathcal{N}(\mu = 0, \sigma^2 = 4)$. Compute the sample mean $\bar{x}$, sample variance s^2 , plot a descriptive histogram, and evaluate the empirical probability $P(X > 1)$.

Q1.b Matrix Operations via Apply

Construct a 10×10 random matrix using the 100 random variables generated in Task 1.a. Use the `apply()` function to calculate the column means across all 10 columns.

Q1.c Logical Vector Subsetting

Filter the random vector from Task 1.a to create a new vector containing only strictly positive values ($x_i > 0$).

2. Algorithmic Iteration & Custom Functions

Q2.a Expectation Approximation via Iterative Loops

Write an R loop to numerically approximate the expected value $\mathbb{E}[X]$ of a Negative Binomial random variable $X \sim \text{NB}(r, p)$ with probability mass function (PMF):

$$P(X = k) = \binom{k-1}{r-1} p^r (1-p)^{k-r} \quad \text{for } k = r, r+1, r+2, \dots$$

Evaluate this numerical approximation across parameters $r \in \{10, 20, 30\}$ paired with success probabilities $p \in \{0.2, 0.5\}$.

Q2.b Function Modularization

Encapsulate the approximation algorithm from Task 2.a into a reusable R function that accepts any target inputs $r > 1$ and $p \in (0, 1)$ and outputs the estimated expectation $\hat{\mathbb{E}}[X]$.

3. Probability Mass Function Visualization

Construct line and scatter plots visualizing the probability mass functions generated in Task 2. Map the realized random variable values ($k = r, r + 1, \dots$) to the x-axis and their corresponding calculated PMF values $P(X = k)$ to the y-axis for each combination of parameters (r, p) .

4. Matrix Heatmaps & Orientation

Use the `image()` function in R to construct a 32-level grayscale heatmap for matrix A :

$$A = \begin{pmatrix} 1 & -1 & 1 \\ 1 & -1 & 1 \\ -1 & 1 & -1 \end{pmatrix}$$

Ensure the color spectrum transitions continuously from white (minimum value) to black (maximum value).

Technical Note on `image()` Coordinate Mapping:

The `image()` function interprets the input matrix Z as values $f(x, y)$ where rows correspond to the x-axis and columns to the y-axis (with column 1 positioned at the bottom). This produces a 90° counter-clockwise rotation relative to standard printed matrix conventions.

5. Hand-Written Digit Recognition Dataset

The USPS Hand-Written Digit Recognition dataset (*LeCun et al., 1990*) consists of 9,298 size-normalized 16×16 grayscale pixel images (7,291 training samples, 2,549 test samples).

The training dataset matrix contains 7,291 rows and 258 columns:

- **Column 1:** Digit Class Label ID (0, 1, ..., 9).
- **Columns 2–257:** Grayscale intensities for 256 pixels (16×16 box, flattened row-wise).
- **Column 258:** Unused / Redundant variable.

Q5.a Data Importation & Cleanup

Import the training dataset into R. Inspect its dimensions and drop the redundant 258th column.

Q5.b Class Distribution Analysis

Generate a frequency table counting the total number of images corresponding to each digit class (0 through 9).

Q5.c Class-Based Matrix Partitioning

Partition the full dataset matrix into 10 separate subset matrices, each containing only observations corresponding to a single digit ID $(0, 1, \dots, 9)$.

Q5.d Digit Rendering

Using the partitioned matrices from Task 5.c and the `image()` function, visualize the first 5 image instances for each digit class (0 to 9).

6. Optimization & Maximum Likelihood Estimation

Consider paired observations $(x_1, y_1), \dots, (x_{100}, y_{100})$ where $x_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 1)$ and responses $y_i \sim \text{Bernoulli}(\pi_i)$ with success probability given by the logistic link function:

$$\pi_i = \frac{\exp(x_i \beta)}{1 + \exp(x_i \beta)}$$

The goal is to estimate parameter β by maximizing the likelihood function:

$$L(\beta) = \prod_{i=1}^{100} \pi_i^{y_i} (1 - \pi_i)^{1-y_i} \implies \ell(\beta) = \sum_{i=1}^{100} [y_i x_i \beta - \log(1 + \exp(x_i \beta))]$$

Data simulation setup in R:

```
set.seed(2025)
x <- rnorm(100)
beta <- 2
pi <- exp(x * beta) / (1 + exp(x * beta))
y <- rbinom(100, size = 1, prob = pi)
```

Q6.a Newton-Raphson Algorithm Formulation

Formulate the Newton-Raphson updating scheme utilizing first and second derivatives (Score $S(\beta)$ and Hessian $H(\beta)$):

$$S(\beta) = \frac{\partial \ell(\beta)}{\partial \beta} = \sum_{i=1}^n x_i (y_i - \pi_i), \quad H(\beta) = \frac{\partial^2 \ell(\beta)}{\partial \beta^2} = - \sum_{i=1}^n x_i^2 \pi_i (1 - \pi_i)$$

$$\beta^{(t+1)} = \beta^{(t)} - \frac{S(\beta^{(t)})}{H(\beta^{(t)})}$$

Q6.b Steepest Ascent Algorithm Formulation

Formulate the Steepest Ascent algorithm incorporating adaptive line-search step halving ($r \leftarrow r/2$) whenever $\ell(\beta^{(t+1)}) < \ell(\beta^{(t)})$:

$$\beta^{(t+1)} = \beta^{(t)} + r \cdot S(\beta^{(t)})$$

Q6.c Gradient Ascent Algorithm Formulation

Formulate the standard Gradient Ascent updating equation with a fixed learning rate $\alpha > 0$:

$$\beta^{(t+1)} = \beta^{(t)} + \alpha \cdot S(\beta^{(t)})$$

Q6.d Algorithmic Implementation & Performance Benchmarking

Implement all three algorithms in R. Compute the final Maximum Likelihood Estimate $\hat{\beta}_{\text{MLE}}$, and compare computational convergence rates, sensitivity to learning hyper-parameters, and relative implementation effort across all three numerical optimization methods.